

Credal Variable Elimination: Exact Marginal Inference for Credal Networks

IBM Research

Abstract

Credal Variable Elimination (CVE) is an exact algorithm for computing lower and upper bounds on marginal probabilities in credal networks. It extends the classical variable elimination algorithm for Bayesian networks to operate on *potentials*—finite sets of real-valued functions representing the extreme points of local credal sets. By combining, marginalizing, and pruning these potentials along an elimination ordering, CVE produces tight probability intervals for any query variable given evidence.

1 Preliminaries

1.1 Credal Networks

A **credal network** is a pair $\langle G, \mathcal{K} \rangle$ where G is a directed acyclic graph (DAG) over variables $\mathbf{X} = \{X_1, \dots, X_n\}$ with finite domains $\mathcal{D} = \{D_1, \dots, D_n\}$, and $\mathcal{K}$ is a collection of conditional credal sets $K(X_i \mid \text{Pa}(X_i) = \pi)$, one for each variable X_i and each configuration π of its parents $\text{Pa}(X_i)$ in G .

Each credal set $K(X_i \mid \pi)$ is a closed convex set of probability distributions over X_i , specified by its *extreme points* (vertices):

$$\text{ext } K(X_i \mid \pi) = \{P_1(X_i \mid \pi), \dots, P_v(X_i \mid \pi)\}.$$

1.2 Strong Extension

The **strong extension** of a separately specified credal network is the convex hull of all joint distributions formed by combining one extreme point per local credal set:

$$\text{co } K(\mathbf{X}) = \text{CH} \left\{ P(\mathbf{x}) = \prod_{i=1}^n P(x_i \mid \pi_i) : P(X_i \mid \pi_i) \in \text{ext } K(X_i \mid \pi_i) \right\}. \quad (1)$$

Theorem 1 (Mauá & Cozman, 2020). *Any combination of extreme points from the local credal sets is an extreme point of the strong extension. Thus the strong extension can be built directly from the local extreme points.*

1.3 Marginal Inference

Given a query variable Z and evidence $\mathbf{Y} = \mathbf{y}$, the goal is to compute:

$$\underline{P}(Z = z \mid \mathbf{Y} = \mathbf{y}) = \min_{P \in \text{ext } K(\mathbf{X})} \frac{\sum_{\mathbf{x}'} \prod_i P(x_i \mid \pi_i)}{\sum_{z'} \sum_{\mathbf{x}'} \prod_i P(x_i \mid \pi_i)}, \quad (2)$$

where the sums in numerator and denominator are over the marginalized variables $\mathbf{X}' = \mathbf{X} \setminus (\{Z\} \cup \mathbf{Y})$, and similarly for the upper probability $\overline{P}$ with max.

2 Potentials

Definition 1 (Potential). A **potential** $\phi(\mathbf{Y})$ over variables $\mathbf{Y}$ is a finite set of non-negative real-valued functions $\{f_1, f_2, \dots, f_m\}$ on the joint domain of $\mathbf{Y}$. Each function f_k maps configurations of $\mathbf{Y}$ to $\mathbb{R}_{\geq 0}$.

Three operations on potentials are central to the algorithm:

Product. The product of $\phi(\mathbf{Y})$ and $\psi(\mathbf{Z})$ is:

$$\phi \cdot \psi = \{f \cdot g : f \in \phi, g \in \psi\},$$

where $(f \cdot g)(\mathbf{y}, \mathbf{z}) = f(\mathbf{y}) \cdot g(\mathbf{z})$ over the joint domain $\mathbf{Y} \cup \mathbf{Z}$ (with broadcasting for non-shared variables).

Sum-Marginal. The sum-marginal of $\phi(\mathbf{Y})$ with respect to $X \subseteq \mathbf{Y}$ is:

$$\sum_X \phi(\mathbf{Y}) = \left\{ \sum_x f(\mathbf{y}) : f \in \phi \right\},$$

producing functions over $\mathbf{Y} \setminus \{X\}$.

Pruning. The pruning operation removes dominated functions. We say g *dominates* f when g is pointwise at least f and strictly greater somewhere:

Definition 2 (Dominance and pruning).

$$\text{prune}(\phi) = \{f \in \phi : \nexists g \in \phi \setminus \{f\} \text{ s.t. } g(\mathbf{y}) \geq f(\mathbf{y}) \forall \mathbf{y} \text{ and } g(\mathbf{y}') > f(\mathbf{y}') \text{ for some } \mathbf{y}'\}.$$

Because a dominated f can only ever attain the min (resp. a dominating g the max) of a quantity that is monotone in the function values, discarding f reduces the cardinality of the potential without affecting the min/max bounds.

Remark 1 (What the implementation does). The strictness requirement “ $g > f$ somewhere” matters, and three properties of `Potential.prune` (`potentials.py:124--148`) are worth recording. (i) Two *identical* functions never dominate each other, so `prune` is not a deduplicator: exact duplicates both survive. (ii) The test is a single greedy double loop that skips functions already marked dominated, both as candidates and as dominators, so the surviving set depends on the enumeration order (it is Pareto-correct, but not a canonical Pareto frontier). (iii) The comparison is made on raw floating-point values with *no* tolerance, so numerically near-identical vertices are retained as distinct functions.

3 Algorithm

CVE answers a query on one variable Z by a single bucket-elimination pass in which Z is eliminated *last*. Algorithm 1 states the pass in full; the paragraphs that follow it explain the three steps that carry the credal content.

One structural fact must be stated first, because it shapes the input and the output. The variables of the credal network are *chain-graph node families*, not the individual atoms of the underlying LCN: a variable X carries a set of atoms $\text{atoms}(X)$ and has domain size $2^{|\text{atoms}(X)|}$.

(the truth table over its atoms, most-significant bit first). A variable with a single atom is called *singleton*; one with several is *compound*. The pass therefore returns bounds on the *states* of Z ; when Z is singleton its state-1 bound is the atom marginal directly, and when Z is compound a further projection is needed to report per-atom marginals (Section 3.2).

Algorithm 1 Credal Variable Elimination (CVE)

Require: Extreme points $\text{ext } \mathcal{K}(X_i \mid \text{pa}(X_i))$ for each variable X_i ; query variable Z ; evidence $\mathbf{Y} = \mathbf{y}$

Ensure: Lower and upper bounds $[\underline{P}(Z=z \mid \mathbf{y}), \overline{P}(Z=z \mid \mathbf{y})]$ for each state z of Z

```

1:  $\Gamma \leftarrow \emptyset$  ▷ the set of potentials
2: for each variable  $X_i \in \mathbf{X}$  do
3:    $\phi_{X_i} \leftarrow \{\text{one chosen } P(X_i \mid \pi) \in \text{ext } \mathcal{K}(X_i \mid \pi) \text{ per parent config } \pi\}$ 
4:    $\Gamma \leftarrow \Gamma \cup \{\phi_{X_i}\}$  ▷  $|\phi_{X_i}| = \prod_{\pi} |\text{ext } \mathcal{K}(X_i \mid \pi)|$  CPT vertices
5: end for
6: for each evidence variable  $Y_j \in \mathbf{Y}$  do
7:    $\Gamma \leftarrow \Gamma \cup \{\{\delta_{y_j}\}\}$  ▷ indicator potential; the CPTs are not sliced
8: end for

9:  $\mathbf{o} \leftarrow (X_{\sigma(1)}, \dots, X_{\sigma(n-1)})$ , an elimination ordering over  $\mathbf{X} \setminus \{Z\}$  ▷ excludes  $Z$ 
10: for each  $X_i$  in ordering  $\mathbf{o}$  do
11:    $\Gamma_{X_i} \leftarrow \{\phi \in \Gamma : X_i \in \text{scope}(\phi)\}$ 
12:    $\Gamma \leftarrow \Gamma \setminus \Gamma_{X_i}$ 
13:   if  $\Gamma_{X_i} = \emptyset$  then
14:     continue
15:   end if
16:    $\psi \leftarrow \prod \{\phi \in \Gamma_{X_i}\}$  ▷ combine all potentials in the bucket
17:    $\psi' \leftarrow \sum_{X_i} \psi$  ▷ marginalize out  $X_i$ ; cardinality unchanged
18:    $\psi'' \leftarrow \text{prune}(\psi')$  ▷ remove dominated functions (Def. 2)
19:    $\Gamma \leftarrow \Gamma \cup \{\psi''\}$ 
20: end for

21:  $\phi_{\text{final}} \leftarrow \prod \{\phi \in \Gamma\}$  ▷ remaining potentials, scope  $\{Z\}$ 
22:  $\underline{P}(z) \leftarrow 1, \overline{P}(z) \leftarrow 0$  for each state  $z$ 
23: for each  $f \in \phi_{\text{final}}$  with  $\sum_{z'} f(z') > 0$  do
24:    $\mathbf{p} \leftarrow f / \sum_{z'} f(z')$  ▷ normalize this vertex
25:   for each state  $z$  do
26:      $\underline{P}(z) \leftarrow \min(\underline{P}(z), \mathbf{p}(z))$ 
27:      $\overline{P}(z) \leftarrow \max(\overline{P}(z), \mathbf{p}(z))$ 
28:   end for
29: end for
30: return  $[\underline{P}(z), \overline{P}(z)]$  for each state  $z$ 

```

Initial potentials are explicit vertex enumerations. Line 3 is where the credal information enters. A family X_i with parent configurations $\pi_1, \dots, \pi_r$ contributes one function per element of $\text{ext } \mathcal{K}(X_i \mid \pi_1) \times \dots \times \text{ext } \mathcal{K}(X_i \mid \pi_r)$: each function is a complete CPT obtained by choosing one extreme point in every column, so $|\phi_{X_i}|$ is the *product* of the per-column vertex counts. The

potential is therefore an explicit enumeration of that family’s contribution to the strong extension, which is what makes CVE a strong-extension method.

The query is eliminated last. The ordering on line 9 omits Z entirely, so no step ever marginalizes the query and its bucket survives to the end. This is the decisive scheduling choice: it is what makes the answer exact with respect to the strong extension, and it is exactly what a single shared two-pass ordering (as in CCTE) gives up. Answering all marginals means repeating the pass once per target, each with its own ordering, since nothing is shared between targets.

Normalization is the evidence conditioning. There is no separate division by $P(\mathbf{y})$. With the indicator potentials of line 7 in the product, a surviving function of ϕ_{final} satisfies $f(z) = P(Z=z, \mathbf{y})$ for one selection of extreme points, so $f(z)/\sum_{z'} f(z') = P(Z=z \mid \mathbf{y})$ for that same selection. Because each vertex is normalized *before* the min/max is taken, the result is the correct set of per-vertex conditionals — not the invalid ratio $\min P(Z, \mathbf{y})/\max P(\mathbf{y})$ of a separately minimized numerator and maximized denominator. Vertices of zero mass are skipped.

3.1 Elimination Ordering

Two heuristics are supported; both exclude the query Z , which is what keeps Z ’s bucket alive.

- **Topological order:** the DAG’s *forward* topological order, restricted to the non-query, non-evidence variables, with the evidence variables appended at the end. Evidence variables are still eliminated — they are pinned by their indicator potentials rather than removed from the problem.
- **Min-fill:** the greedy interaction-graph heuristic, eliminating at each step the variable that adds the fewest fill-in edges, with an alphabetical tiebreak, and with Z held out of the ordering (it still counts as a neighbour when fill is scored). This aims at a small induced width w^* .

3.2 Projecting a Compound Query onto its Atoms

When Z is singleton, its atom marginal is the state-1 entry of the bounds returned by Algorithm 1. When Z is compound, with m atoms and $k = 2^m$ states, each atom a is recovered from the state bounds $[\underline{P}(z), \bar{P}(z)]$ by two linear programs over the states S_a in which a is true:

$$\underline{P}(a) = \min \left\{ \sum_{z \in S_a} p_z : \sum_z p_z = 1, \underline{P}(z) \leq p_z \leq \bar{P}(z) \right\},$$

and symmetrically $\bar{P}(a)$ with max. The feasible region is the box $\prod_z [\underline{P}(z), \bar{P}(z)]$ intersected with the probability simplex, which is an *outer relaxation* of the true state polytope of Z . Atom bounds on a compound query can therefore be looser than the state bounds CVE actually computed; the running example in Section 4 shows this at its most extreme.

4 Running Example

Example 1 (Alarm Network). The LCN `examples/alarm.lcn` over atoms A, B, C, D, E factorizes into four credal-network nodes: the singletons B (Burglary), E (Earthquake), A (Alarm), and the *compound* node $C-D$ carrying the two atoms C, D (they are merged because the sentence

$0.5 \leq P(\neg(C \oplus D) \mid A) \leq 0.7$ constrains them jointly). The compound node has $2^2 = 4$ states, ordered $CD \in \{00, 01, 10, 11\}$ most-significant bit first. The DAG is

$$B \rightarrow A, \quad E \rightarrow A, \quad A \rightarrow C-D.$$

The local credal sets (extreme points from LRS vertex enumeration) are:

Node	Parent Config	#	Vertices
B	—	2	$[0.8, 0.2], [0.9, 0.1]$
E	—	2	$[0.9, 0.1], [0.95, 0.05]$
A	$B=0, E=0$	2	$[0.956, 0.044], [1.0, 0.0]$
A	$B=1, E=0$	2	$[0.0, 1.0], [1.0, 0.0]$
A	$B=0, E=1$	2	$[0.0, 1.0], [1.0, 0.0]$
A	$B=1, E=1$	2	$[0.0, 1.0], [1.0, 0.0]$
$C-D$	$A=0$	4	$[0, 0, 0, 1], [0, 0, 1, 0], [0, 1, 0, 0], [1, 0, 0, 0]$
$C-D$	$A=1$	11	$[0, 0, 0.3, 0.7], [0, 0.3, 0, 0.7], \dots, [0.7, 0, 0, 0.3]$

Initial potentials. Line 3 of Algorithm 1 takes one extreme point per parent configuration, so the potential sizes are the *products* of the counts above:

$$|\phi_B| = 2, \quad |\phi_E| = 2, \quad |\phi_A| = 2^4 = 16, \quad |\phi_{C-D}| = 4 \times 11 = 44,$$

for $2 + 2 + 16 + 44 = 64$ functions in total across the four potentials.

Query: $P(B=1)$ (**no evidence**). The forward topological order with the target removed is $\mathbf{o} = (E, A, C-D)$; note B never appears in it.

Step 1: eliminate E . The bucket is ϕ_E (scope $\{E\}$, 2 functions) and ϕ_A (scope $\{A, B, E\}$, 16 functions). Their product has $2 \times 16 = 32$ functions over $\{A, B, E\}$; summing out E leaves 32 functions over $\{A, B\}$, and pruning removes none of them.

Step 2: eliminate A . The bucket is the new potential over $\{A, B\}$ (32 functions) together with ϕ_{C-D} (scope $\{C-D, A\}$, 44 functions), giving $32 \times 44 = 1408$ functions over $\{C-D, A, B\}$. Summing out A leaves 1408 functions over $\{C-D, B\}$; again none are dominated.

Step 3: eliminate $C-D$. Summing out $C-D$ leaves 1408 functions over $\{B\}$, and here pruning bites: only 328 survive.

Result. Each surviving f over $\{B\}$ is normalized, $p(B=1) = f(1)/(f(0) + f(1))$, and the componentwise min/max over the 328 normalized vectors gives

$$\underline{P}(B=1) = 0.10, \quad \overline{P}(B=1) = 0.20,$$

recovering exactly the interval asserted by the sentence $0.1 \leq P(B) \leq 0.2$, as it must since B is a root.

The compound node illustrates the projection gap. Running the driver over all four nodes returns $P(A=1) \in [0, 0.312]$ and $P(E=1) \in [0.05, 0.10]$, but the state bounds for $C-D$ come out vacuous, $[\underline{p}, \overline{p}] = [0, 1]$ on all four states. The per-atom linear programs of Section 3.2 can then only return $P(C=1), P(D=1) \in [0, 1]$: with every state interval equal to $[0, 1]$, the box-simplex relaxation carries no information, even though the joint constraint on $C \oplus D$ does. This is the outer relaxation of the compound-node projection at its worst.

5 Succinct Description (paper-ready)

Credal Variable Elimination (CVE) extends classical bucket elimination (Dechter, 1999) to credal networks by replacing every factor with a *potential*: a finite set of non-negative functions over a common scope, each member being one extreme point of the corresponding local credal set. Potentials support three operations. Their *product* is the elementwise product of every pair, $\phi \cdot \psi = \{f \cdot g : f \in \phi, g \in \psi\}$, so cardinalities multiply; the *sum-marginal* $\sum_X \phi = \{\sum_x f : f \in \phi\}$ sums each member separately and leaves the cardinality unchanged; and *pruning* discards any f for which some $g \in \phi$ satisfies $g \geq f$ pointwise with $g > f$ somewhere, which shrinks the set without affecting its extremes, since a dominated function attains neither. To answer a query on Z under evidence $\mathbf{Y} = \mathbf{y}$, CVE builds one potential per family — enumerating the conditional probability tables obtained by choosing one extreme point in each parent configuration, so the potential explicitly lists that family’s contribution to the strong extension — adds a single-function indicator potential for each observation, and then eliminates the variables one at a time along an ordering that *excludes* Z : at each step it multiplies together all potentials whose scope contains the current variable, sums that variable out, and prunes the result.

Because Z is never eliminated, its bucket survives to the end, and the final product is a potential over Z alone. Each of its members equals $P(Z, \mathbf{y})$ under one selection of extreme points, so dividing that member by its own sum yields $P(Z \mid \mathbf{y})$ for the same selection; taking the componentwise minimum and maximum *after* normalizing each member therefore gives bounds that are exact with respect to the strong extension — and not the invalid ratio of a separately minimized numerator and a separately maximized denominator. Never eliminating the query is what buys this exactness, and it is precisely what a single shared two-pass ordering gives up; the price is that bounds for all variables require one pass per target, with nothing reused between them. The cost of a pass is $O(n \cdot C \cdot k^{w^*})$ for n variables of domain size at most k , an ordering of induced width w^* , and C bounding the cardinality of the intermediate potentials. Pruning is what keeps C in check: it can still grow exponentially in the number of extreme points in the worst case, but remains manageable in practice on networks of moderate induced width.

6 Complexity

Theorem 2. *The time and space complexity of CVE is $O(n \cdot C \cdot k^{w^*})$, where n is the number of variables, k bounds the domain sizes, w^* is the induced width of the elimination ordering, and C bounds the cardinality of the intermediate potentials.*

In the worst case, C is exponential in the number of extreme points. However, pruning keeps C manageable in practice for networks of moderate treewidth.

7 References

1. D.D. Mauá, F.G. Cozman. Thirty years of credal networks: Specification, algorithms and complexity. *International Journal of Approximate Reasoning*, 126:133–157, 2020.
2. R. Marinescu, R. Dechter, A. Ihler. Credal Marginal MAP. *NeurIPS*, 2023.
3. F.G. Cozman. Credal networks. *Artificial Intelligence*, 120(2):199–233, 2000.
4. R. Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113(1–2):41–85, 1999.

5. D.D. Mauá, C.P. de Campos, A. Benavoli, A. Antonucci. Probabilistic inference in credal networks: New complexity results. *Journal of Artificial Intelligence Research*, 50:603–637, 2014.